Brain MRI Tumor Segmentation

Model Development, Training & Evaluation Report

Dataset: BRISC 2025 (Segmentation Task)

Scope: Deep learning architecture, training pipeline, final model, and evaluation

Purpose

This report documents the modeling work performed after the BRISC data pipeline handoff. It records the baseline U-Net experiment, the transition to a pretrained ResNet34 encoder, the training and validation strategy, evaluation metrics, final results, and the structure of the final segmentation model.

Contents

1. Project Scope and Modeling Objective

The modeling stage focused on binary brain-tumor segmentation using the BRISC 2025 segmentation task. The objective was to train a segmentation model that receives a preprocessed single-channel MRI image and predicts a binary tumor mask (background = 0, tumor = 1).

Item	Final modeling choice
Task	Binary tumor-vs-background segmentation
Dataset	BRISC 2025 Segmentation Task
Input modality representation	Single-channel grayscale MRI
Working image size	256 x 256 pixels
Train/validation split	80% / 20% stratified from the BRISC training split
Test split	Official BRISC test split, kept untouched
Final architecture	U-Net decoder with pretrained ResNet34 encoder
Final checkpoint	best_resnet34_unet.pth

2. Data Pipeline Contract Used by the Model

The modeling notebook consumed the companion data pipeline rather than rebuilding its preprocessing logic. The handoff report documented 3,933 valid train image/mask pairs and 860 valid test pairs, with no corrupted images detected. The raw dataset contains mixed image resolutions and both grayscale and RGB files, while image/mask dimensions are paired correctly.

Property	Value used in modeling
Train samples	3,933
Test samples	860
Image tensor	float32, [1, 256, 256]
Mask tensor	int64, [256, 256]
Mask values	{0, 1}
Image normalization	mean = 0.5, std = 0.5
Mask threshold	127
Train augmentation	Horizontal flip + vertical flip + rotation up to +/-15 degrees
Validation/test augmentation	Disabled/deterministic

The data report identified important raw-mask noise near the 0 and 255 extremes and therefore binarized masks at threshold 127 before tensor conversion. It also documented that spatial resolution is heterogeneous and that dropping small outliers would disproportionately remove meningioma and pituitary samples; the pipeline therefore retained all valid samples and resized them to a common working resolution.

3. Experimental Setup

The model was trained in Google Colab using a Tesla T4 GPU.

Component	Final setting
Hardware	Tesla T4 GPU
Framework	PyTorch
Image size	256 x 256
Batch size	8
Primary loss	50% BCE + 50% Dice
Optimizer	AdamW
Learning-rate scheduler	ReduceLROnPlateau on validation Dice
Early stopping	Enabled; patience configured in notebook
Model selection	Best validation Dice
Inference threshold	0.5 for standard reported metrics

4. Baseline U-Net

A standard U-Net was first implemented as the baseline. The encoder used four convolutional stages followed by a bottleneck; the decoder progressively upsampled features with transposed convolutions and skip connections from the corresponding encoder stages. This baseline established a reference point under the same 256 x 256 input pipeline, loss, and training protocol.

Metric	Baseline U-Net test result
Dice	0.8540
IoU	0.7796
Precision	0.8717
Recall	0.8704
Loss	0.0861

The training and validation curves were inspected before changing the architecture. Both Dice curves increased over training and remained relatively close, indicating that the baseline was not exhibiting a severe train-validation divergence.

5. ResNet34 U-Net Architecture

The final model, named ResNet34UNet, preserves the U-Net decoder concept while replacing the conventional convolutional encoder with an ImageNet-pretrained ResNet34 backbone. This was selected as a targeted architectural change intended to improve feature extraction without changing the rest of the pipeline.

Encoder feature	Approx. resolution	Channels	Decoder role
x1	1/2 of input	64	First skip connection
x2	1/4	64	Second skip connection
x3	1/8	128	Third skip connection
x4	1/16	256	Fourth skip connection
x5	1/32	512	Bottleneck input

Decoder flow: x5 -> dec4 with skip x4 -> dec3 with skip x3 -> dec2 with skip x2 -> dec1 with skip x1 -> final transposed convolution -> final convolution -> one-channel segmentation logit map.

Stage	Operation	Output role
Encoder	ResNet34 stem + layer1-4	Hierarchical image features
Bottleneck	512-channel deepest features	Compact semantic representation
Decoder block 4	Upsample 512 -> 256 + x4	Recover 1/16-scale details
Decoder block 3	Upsample 256 -> 128 + x3	Recover 1/8-scale details
Decoder block 2	Upsample 128 -> 64 + x2	Recover 1/4-scale details
Decoder block 1	Upsample 64 -> 64 + x1	Recover 1/2-scale details
Final head	Upsample 64 -> 32 -> 1 channel	Binary tumor logits

6. How ResNet34 Was Integrated into U-Net

The integration consists of three main modifications: (1) replacing the original U-Net encoder with ResNet34, (2) adapting ResNet34 from RGB input to one-channel MRI input, and (3) connecting ResNet34 intermediate feature maps to the U-Net decoder through skip connections.

6.1 One-channel MRI adaptation

The torchvision ResNet34 backbone is normally defined for three-channel RGB images. Because the MRI pipeline produces a single grayscale channel, the original conv1 layer was replaced with a one-channel convolution having the same output channel count, kernel size, stride, and padding. The pretrained RGB filters were converted to one-channel filters by averaging the original weights across the input-channel dimension. This preserves a direct initialization derived from the pretrained weights while making the backbone compatible with [1, 256, 256] MRI tensors.

```
original_conv = backbone.conv1

new_conv = nn.Conv2d(
    in_channels=1,
    out_channels=original_conv.out_channels,
    kernel_size=original_conv.kernel_size,
    stride=original_conv.stride,
    padding=original_conv.padding,
    bias=False
)
```

```
with torch.no_grad():
    new_conv.weight.copy_(
        original_conv.weight.mean(dim=1, keepdim=True)
    )

backbone.conv1 = new_conv
```

6.2 Skip-connection wiring

The encoder returns five feature tensors (x1-x5). The deepest feature x5 is passed to the first decoder block. Each decoder block upsamples the current representation and concatenates it with the matching encoder feature map before applying two convolutional layers. This is the same core information flow that makes U-Net effective at recovering spatial detail while using deep semantic features.

```
x1, x2, x3, x4, x5 = self.encoder(x)

d4 = self.dec4(x5, x4)
d3 = self.dec3(d4, x3)
d2 = self.dec2(d3, x2)
d1 = self.dec1(d2, x1)

out = self.final_up(d1)
out = self.final_conv(out)
```

6.3 Decoder block

Each DecoderBlock uses a transposed convolution for 2x upsampling, concatenates the corresponding skip tensor along the channel dimension, then applies two 3x3 convolutions with BatchNorm and ReLU. The final head maps the 32-channel representation to one output channel of segmentation logits.

7. Loss Function and Optimization

The final training objective used a balanced BCE + Dice formulation. BCE provides pixel-wise classification supervision, while Dice loss directly encourages overlap between predicted tumor regions and ground-truth masks. This combination is appropriate for binary segmentation where foreground pixels are substantially less frequent than background pixels.

Component	Configuration
BCE loss	BCEWithLogitsLoss
Dice loss	Soft sigmoid Dice loss
Combined loss	0.5 * BCE + 0.5 * Dice
Optimizer	AdamW
Initial learning rate	1e-4
Weight decay	1e-5
Scheduler	ReduceLROnPlateau, monitored on validation Dice

8. Training and Validation Strategy

The official BRISC training split was divided into 80% training and 20% validation (3,146 / 787 samples) using a fixed random seed and stratification by tumor type encoded in filenames (gl, me, pi). The official BRISC test split (860 samples) was kept untouched and used only for final evaluation.

Split	Samples	Augmentation	Purpose
Train	3,146	Enabled	Optimize model parameters
Validation	787	Disabled	Model selection, scheduler, early stopping
Test	860	Disabled	Final one-time evaluation

The train/validation class proportions remained essentially unchanged after stratification: glioma approximately 29.2%, meningioma approximately 33.8%, and pituitary approximately 37.0%. The batch interface was verified as [8, 1, 256, 256] for images and [8, 256, 256] for masks.

9. Evaluation Protocol

The evaluation pipeline was expanded beyond the core overlap metrics to include classification-style pixel metrics, boundary distance, and threshold analysis. All standard reported metrics below use a 0.5 sigmoid threshold unless explicitly stated otherwise.

Metric / output	Implemented
Dice Score	Yes
IoU	Yes
Precision	Yes
Recall / Sensitivity	Yes
Accuracy	Yes
Specificity	Yes
HD95	Yes; reported in pixels
Precision-Recall Curve	Yes
Dice vs Threshold	Yes
Train / Validation Loss curves	Yes
Train / Validation Dice curves	Yes
Train / Validation IoU curves	Yes

HD95 is computed in image-pixel coordinates because the pipeline operates on 2D PNG slices and does not retain physical voxel spacing metadata. Therefore, the reported HD95 should be labeled in pixels, not millimeters.

10. Quantitative Results

10.1 Baseline vs final architecture

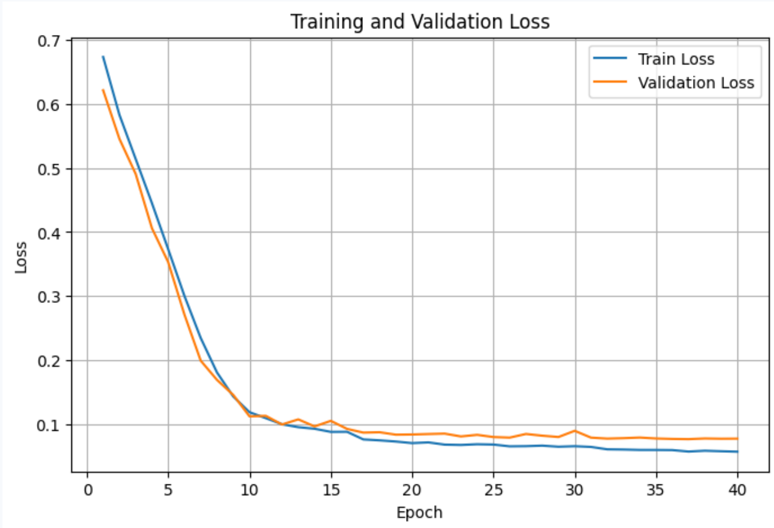
Metric	Baseline U-Net	ResNet34-U-Net	Change
Dice	0.8540	0.8731	+0.0191
IoU	0.7796	0.8042	+0.0246
Precision	0.8717	0.8793	+0.0076
Recall	0.8704	0.8958	+0.0254
Loss	0.0861	0.0741	-0.012

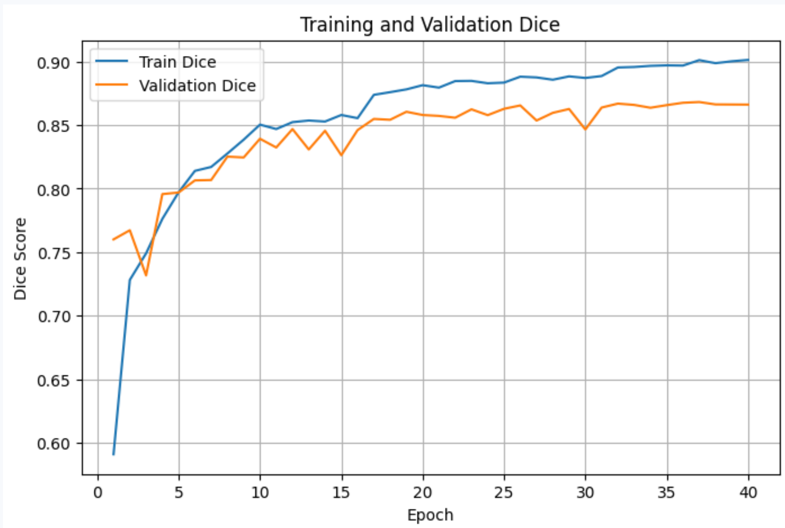
The pretrained ResNet34 encoder improved every reported baseline metric above, with the largest absolute gains in Dice, IoU, and Recall. The final reported test Dice of 0.8731 is therefore the main headline performance result of the modeling stage.

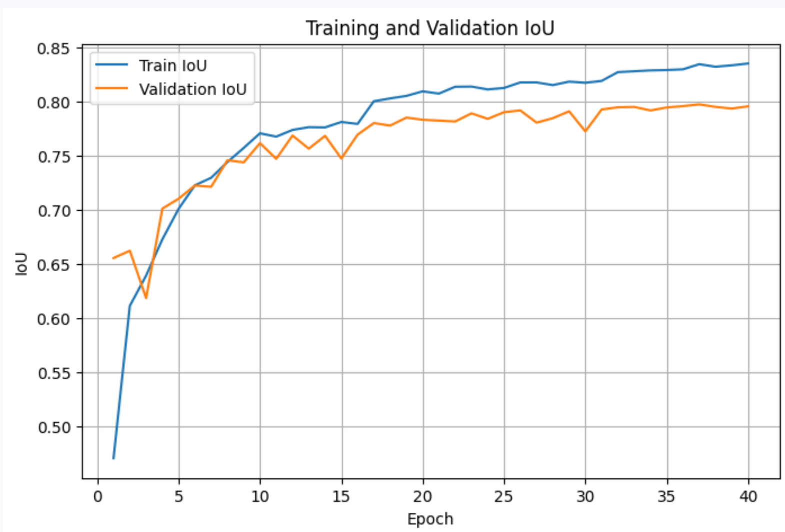
10.2 Final evaluation

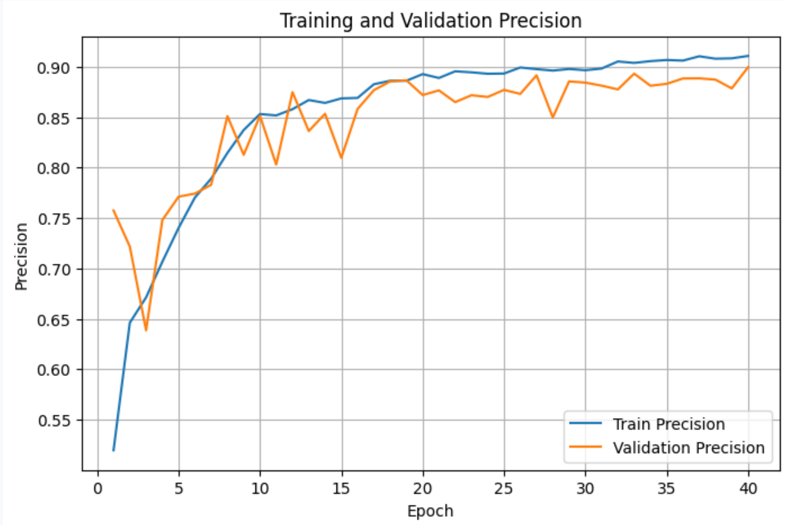
Final metric	Value
Dice Score	0.8731
IoU	0.8042
Precision	0.8793
Recall	0.8958
Accuracy	0.9964
Specificity	0.9982
HD95 (pixels)	9.5113
PR-AUC	0.9508
Best Threshold	0.50
Best Dice at Selected Threshold	0.9051

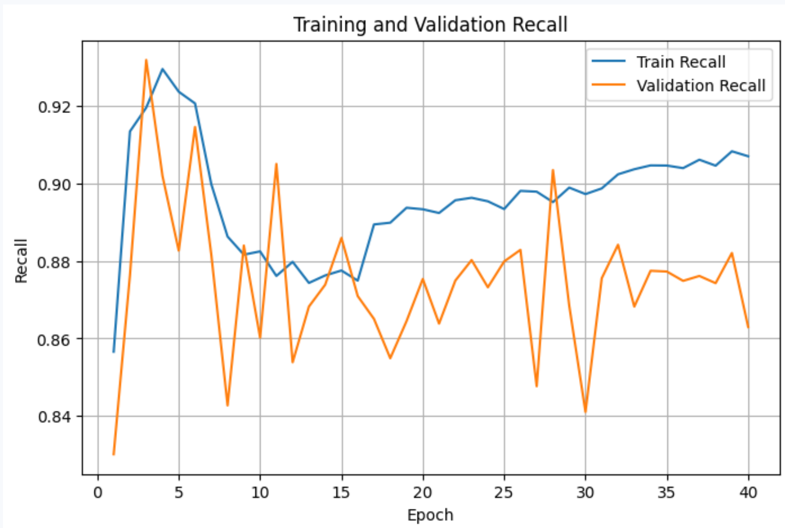
10.3 Training and validation figures

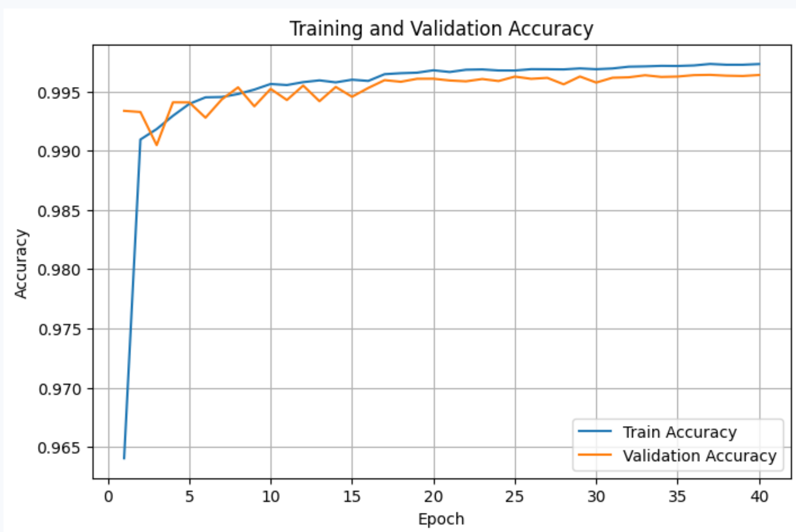


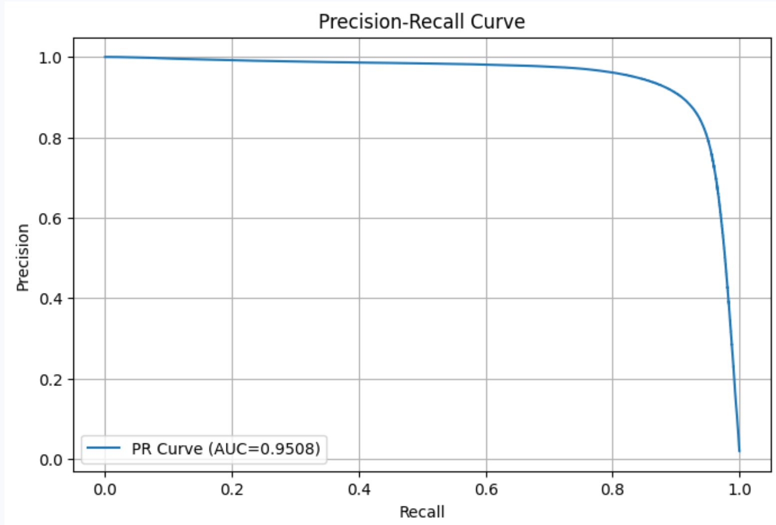


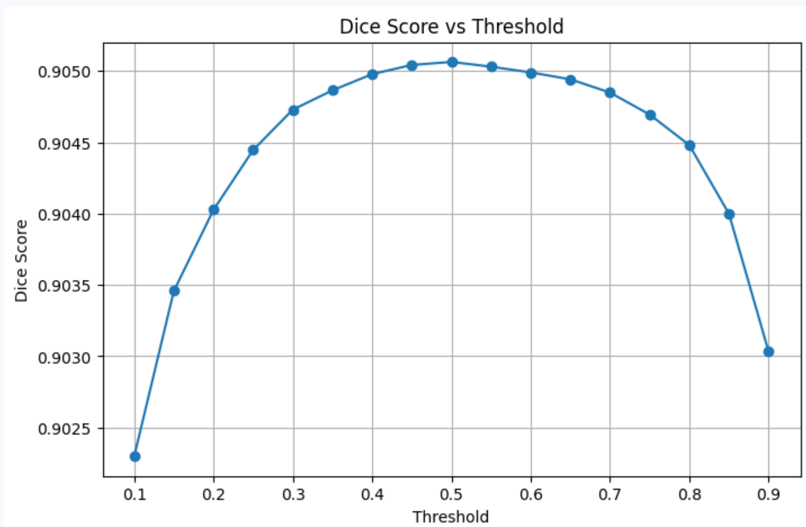



Training and Validation Precision
Precision
0.90
0.85
0.80
0.75
0.70
0.65
0.60
0.55
0
5
10
15
20
25
30
35
40
Epoch
Train Precision
Validation Precision

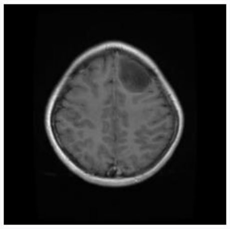
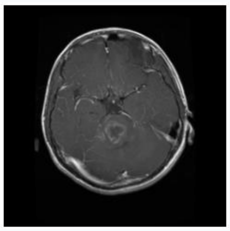
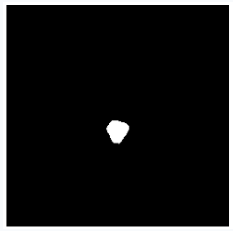
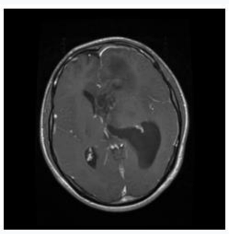

Training and Validation Recall
Recall
0.92
0.90
0.88
0.86
0.84
0
5
10
15
20
25
30
35
40
Epoch
Train Recall
Validation Recall


Training and Validation Accuracy
Accuracy
0.995
0.990
0.985
0.980
0.975
0.970
0.965
0
5
10
15
20
25
30
35
40
Epoch
Train Accuracy
Validation Accuracy


Precision-Recall Curve
Precision
Recall
PR Curve (AUC=0.9508)
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


Dice Score vs Threshold
Dice Score
Threshold
0.9050
0.9045
0.9040
0.9035
0.9030
0.9025
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9

11. Qualitative Results

Original MRI Image	Ground Truth Mask	Predicted Mask
		
		
		

Appendix. Model Implementation Reference

The final model implementation contains three required classes. Their dependency order is: ResNet34Encoder -> DecoderBlock -> ResNet34UNet.

Class	Responsibility
ResNet34Encoder	Loads ImageNet-pretrained ResNet34, adapts conv1 to one channel, and returns x1-x5 feature maps.
DecoderBlock	Upsamples a decoder feature map, concatenates the matching encoder skip feature, and applies convolutional refinement.
ResNet34UNet	Connects the encoder and decoder, performs final upsampling, and returns one-channel segmentation logits.

Parameter count reported by the notebook after model construction: approximately 24.44 million trainable parameters. Forward-pass verification produced an output tensor of [8, 1, 256, 256] for an input batch of [8, 1, 256, 256].